Compiler Design Lab

Date: 6-3-2026 **Name:** S Vishwajith
Assignment No.: 9 **Register No.:** 23BCE1145

Aim:

To implement a code to find Three Address code and it's 3 data structures using C++.

Code:

```
#include <iostream>

#include <stack>

#include <vector>


using namespace std;

struct token

{

   string c;

   int type; // 0-id, 1-unary, 2-binary, 3-equals

};

struct statement

{

   int temp; // 0-none, 1-arg1, 2-arg2, 3-arg1&&arg2

   int eq;

   string label;

   string res;

   string op;

   string arg1;

   string arg2;
```

```
};

vector<token> input;
vector<statement> tac;
int count = 0;
int ifcount = 0;

void to_tac()
{
    stack<token> s;
    stack<int> ifstack;
    string lab = "";
    int ind = 0;
    for (token t : input)
    {
        ind++;
        if (t.type == 0 && t.c == "if")
        {
            statement st;
            st.op = "if True goto";
            st.arg1 = s.top().c;
            st.temp = 0;
            st.eq = 2;
            if (st.arg1[0] == 't' && st.arg1[1] <= '9' && st.arg1[1] >= '0')
                st.temp = 1;
            st.arg2 = "L" + to_string(ifcount);
            lab = "L" + to_string(ifcount++);
            st.res = "";
```

```
    tac.push_back(st);
    statement st1;
    st1.eq = 3;
    st1.temp = 0;
    st1.op = "goto";
    st1.arg1 = "L";
    st1.arg2 = "";
    st1.res = "";
    tac.push_back(st1);
    ifstack.push(tac.size() - 1);
}
else if (t.type == 0 && t.c == "else")
{
    statement st1;
    st1.eq = 3;
    st1.temp = 0;
    st1.op = "goto";
    st1.arg1 = "L";
    st1.arg2 = "";
    st1.res = "";
    tac.push_back(st1);
    lab = "L" + to_string(ifcount);
    tac[ifstack.top()].arg1 += to_string(ifcount++);
    ifstack.pop();
    ifstack.push(tac.size() - 1);
}
else if (t.type == 0 && t.c == "endif")
{
```

```
    lab = "L" + to_string(ifcount);
    if (ind == input.size())
    {
        statement st;
        st.eq = 5;
        st.label = "L" + to_string(ifcount);
        tac.push_back(st);
        while (!s.empty())
        {
            string lb = tac[ifstack.top()].arg1;
            if (lb == "L")
                tac[ifstack.top()].arg1 += to_string(ifcount);
            s.pop();
        }
    }
}
else if (t.type == 0)
    s.push(t);
else if (t.type == 1)
{
    token arg = s.top();
    s.pop();
    statement st;
    st.temp = 0;
    st.eq = 0;
    st.res = "t" + to_string(::count);
    st.op = t.c;
    st.arg1 = arg.c;
```

```
    st.arg2 = "";
    if (st.arg1[0] == 't' && st.arg1[1] <= '9' && st.arg1[1] >= '0')
        st.temp = 1;
    if (lab != "")
    {
        st.label = lab;
        lab = "";
    }
    tac.push_back(st);
    token n;
    n.type = 0;
    n.c = "t" + to_string(::count++);
    s.push(n);
}
else if (t.type == 2)
{
    token arg2 = s.top();
    s.pop();
    token arg1 = s.top();
    s.pop();
    statement st;
    st.temp = 0;
    st.eq = 0;
    st.res = "t" + to_string(::count);
    st.op = t.c;
    st.arg1 = arg1.c;
    st.arg2 = arg2.c;
    if (st.arg1[0] == 't' && st.arg1[1] <= '9' && st.arg1[1] >= '0')
```

```
        st.temp += 1;
    if (st.arg2[0] == 't' && st.arg2[1] <= '9' && st.arg2[1] >= '0')
        st.temp += 2;
    if (lab != "")
    {
        st.label = lab;
        lab = "";
    }
    tac.push_back(st);
    token n;
    n.type = 0;
    n.c = "t" + to_string(::count++);
    s.push(n);
}
else if (t.type == 3)
{
    token arg1 = s.top();
    s.pop();
    token arg2 = s.top();
    s.pop();
    statement st;
    st.arg1 = arg2.c;
    st.op = "=";
    st.arg2 = arg1.c;
    st.eq = 1;
    st.res = "";
    if (lab != "")
    {
```

```
            st.label = lab;
            lab = "";
        }
        tac.push_back(st);
    }
  }
}

int find_temp(string temp)
{
  int i = 0;
  for (statement s : tac)
  {
    if (s.res == temp || (s.arg1 == temp && s.eq == 1))
        return i;
    i++;
  }
  return -1;
}

void print_tac()
{
  cout << endl
     << "Three Address Code: " << endl;
  for (statement s : tac)
  {
    if (s.label == "")
        cout << "    ";
```

```
        else
            cout << s.label << ": ";
        if (s.eq == 1)
            cout << s.arg1 << " = " << s.arg2;
        else if (s.eq == 2)
            cout << "if " << s.arg1 << " True goto " << s.arg2;
        else if (s.eq == 3)
            cout << "goto " << s.arg1;
        else if (s.eq == 5)
            cout << " ";
        else if (s.arg2 == "")
            cout << s.res << " = " << s.op << " " << s.arg1;
        else
            cout << s.res << " = " << s.arg1 << " " << s.op << " " << s.arg2;
        cout << endl;
    }
}

void print_quad()
{
    cout << "\nQuadruples: \n\n";
    cout << "|    OP    |  ARG1  |  ARG2  |   RES  |" << endl;
    cout << "-----------------------------------------------" << endl;
    for (statement s : tac)
    {
        if (s.eq == 5)
            continue;
        string pad;
```

```
        cout << '|';
        string out = s.op;
        pad = string(12 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.arg1;
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.arg2;
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.res;
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        cout << endl;
    }
}

void print_trip()
{
    cout << "\nTriples: \n\n";
    cout << "|   INDEX  |    OP    |  ARG1  |   ARG2  |" << endl;
    cout << "-----------------------------------------------" << endl;
    int i = 0;
    for (statement s : tac)
    {
        if (s.eq == 5)
            continue;
        string pad;
```

```
        string out;
        cout << '|';
        out = to_string(i);
        pad = string(12 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.op;
        pad = string(12 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.arg1;
        if (out[0] == 't' && out.size() > 1)
            out = '[' + to_string(find_temp(out)) + ']';
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        out = s.arg2;
        if (out[0] == 't' && out.size() > 1)
            out = '[' + to_string(find_temp(out)) + ']';
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        cout << endl;
        i++;
    }
}

void print_i_trip()
{
    cout << "\nIndirect Triples: \n\n";
    cout << "|  ADDR  |    INDEX   |  |    INDEX   |     OP     |  ARG1   |   ARG2  |" << endl;
    cout << "------------------------   ----------------------------------------------------" << endl;
```

```
int i = 0;
int a = 31;
for (statement s : tac)
{
   if (s.eq == 5)
      continue;
   string pad;
   string out;
   cout << '|';
   out = to_string(a);
   pad = string(8 - out.size(), ' ');
   cout << out << pad << '|';
   out = to_string(i);
   pad = string(12 - out.size(), ' ');
   cout << out << pad << '|';
   cout << "   |";
   out = to_string(i);
   pad = string(12 - out.size(), ' ');
   cout << out << pad << '|';
   out = s.op;
   pad = string(12 - out.size(), ' ');
   cout << out << pad << '|';
   out = s.arg1;
   if (out[0] == 't' && out.size() > 1)
      out = '[' + to_string(a - i + find_temp(out)) + ']';
   pad = string(10 - out.size(), ' ');
   cout << out << pad << '|';
   out = s.arg2;
```

```
        if (out[0] == 't' && out.size() > 1)
            out = '[' + to_string(a - i + find_temp(out)) + ']';
        pad = string(10 - out.size(), ' ');
        cout << out << pad << '|';
        cout << endl;
        i++;
        a++;
    }
}

int main()
{
    string in, var;
    int un = 0;
    cin >> in;
    cout << "INPUT: " << in << endl;
    for (char c : in)
    {
        if (c == ',')
        {
            if (!var.empty())
            {
                token n;
                n.type = 0;
                n.c = var;
                input.push_back(n);
                var.clear();
            }
```

```
    un = 0;
}
else if (c == 'u')
{
    un = 1;
    var.push_back(c);
}
else if (c == '-' && un == 1)
{
    token n;
    n.type = 1;
    n.c = '-';
    input.push_back(n);
    var.clear();
    un = 0;
}
else if (c == '=')
{
    token n;
    n.type = 3;
    n.c = '=';
    input.push_back(n);
    un = 0;
    var.clear();
}
else if (('a' <= c && c <= 'z') || ('0' <= c && c <= '9') || ('A' <= c && c <= 'Z'))
{
    var.push_back(c);
```

```
            un = 0;
        }
        else if (c == '+' || c == '-' || c == '%' || c == '*' || c == '/' || c == '^' || c == '<')
        {
            token n;
            n.type = 2;
            n.c = c;
            input.push_back(n);
            var.clear();
            un = 0;
        }
    }
    if (!var.empty())
    {
        token n;
        n.type = 0;
        n.c = var;
        input.push_back(n);
        var.clear();
    }

    to_tac();
    print_tac();
    print_quad();
    print_trip();
    print_i_trip();
}
```

Inputs and Outputs:

Input 1: p = q * -r + q – r + (t * u + v * w)

Postfix: p,q,r,u-,*,q,+,r,-,t,u,*,v,w,*,+,+,=

Output:

```
PS C:\Users\vishw\Coding\Compiler-Lab\Week-10> .\TAC
p,q,r,u-,*,q,+,r,-,t,u,*,v,w,*,+,+,=
INPUT: p,q,r,u-,*,q,+,r,-,t,u,*,v,w,*,+,+,=

Three Address Code:
    t0 = - r
    t1 = q * t0
    t2 = t1 + q
    t3 = t2 - r
    t4 = t * u
    t5 = v * w
    t6 = t4 + t5
    t7 = t3 + t6
    p = t7

Quadruples:

|     OP     |   ARG1   |   ARG2   |    RES   |
-----------------------------------------------
|-           |r         |          |t0        |
|*           |q         |t0        |t1        |
|+           |t1        |q         |t2        |
|-           |t2        |r         |t3        |
|*           |t         |u         |t4        |
|*           |v         |w         |t5        |
|+           |t4        |t5        |t6        |
|+           |t3        |t6        |t7        |
|=           |p         |t7        |          |
```

Triples:

INDEX	OP	ARG1	ARG2
0	-	r	
1	*	q	[0]
2	+	[1]	q
3	-	[2]	r
4	*	t	u
5	*	v	w
6	+	[4]	[5]
7	+	[3]	[6]
8	=	p	[7]

Indirect Triples:

ADDR	INDEX
31	0
32	1
33	2
34	3
35	4
36	5
37	6
38	7
39	8

INDEX	OP	ARG1	ARG2
0	-	r	
1	*	q	[31]
2	+	[32]	q
3	-	[33]	r
4	*	t	u
5	*	v	w
6	+	[35]	[36]
7	+	[34]	[37]
8	=	p	[38]

Input 2: x = (m+(n^o) / ((p-q)/(r+(n^o)) % s/t))

Postfix: x,m,n,o,^,p,q,-,r,n,o,^,+,/,s,t,/,%,/,=

Output:

```
x,m,n,o,^,p,q,-,r,n,o,^,+,/,s,t,/,%,/,=
INPUT: x,m,n,o,^,p,q,-,r,n,o,^,+,/,s,t,/,%,/,=

Three Address Code:
    t0 = n ^ o
    t1 = p - q
    t2 = n ^ o
    t3 = r + t2
    t4 = t1 / t3
    t5 = s / t
    t6 = t4 % t5
    t7 = t0 / t6
    m = t7

Quadruples:

|     OP     |   ARG1   |   ARG2   |    RES   |
-------------------------------------------------
|^           |n         |o         |t0        |
|-           |p         |q         |t1        |
|^           |n         |o         |t2        |
|+           |r         |t2        |t3        |
|/           |t1        |t3        |t4        |
|/           |s         |t         |t5        |
|%           |t4        |t5        |t6        |
|/           |t0        |t6        |t7        |
|=           |m         |t7        |          |
```

Triples:

INDEX	OP	ARG1	ARG2
0	^	n	o
1	-	p	q
2	^	n	o
3	+	r	[2]
4	/	[1]	[3]
5	/	s	t
6	%	[4]	[5]
7	/	[0]	[6]
8	=	m	[7]

Indirect Triples:

ADDR	INDEX
31	0
32	1
33	2
34	3
35	4
36	5
37	6
38	7
39	8

INDEX	OP	ARG1	ARG2
0	^	n	o
1	-	p	q
2	^	n	o
3	+	r	[33]
4	/	[32]	[34]
5	/	s	t
6	%	[35]	[36]
7	/	[31]	[37]
8	=	m	[38]

Input 3: if x<y, p=p+1 else q=q+1 endif

Postfix: x,y,<,if,p,p,1,+,=,else,q,q,1,+,=,endif

Output:

```
PS C:\Users\vishw\Coding\Compiler-Lab\Week-10> .\TAC
x,y,<,if,p,p,1,+,=,else,q,q,1,+,=,endif
INPUT: x,y,<,if,p,p,1,+,=,else,q,q,1,+,=,endif

Three Address Code:
    t0 = x < y
    if t0 True goto L0
    goto L1
L0: t1 = p + 1
    p = t1
    goto L2
L1: t2 = q + 1
    q = t2
L2:

Quadruples:

|     OP      |   ARG1   |   ARG2   |    RES   |
-----------------------------------------------
|<            |x         |y         |t0        |
|if True goto|t0         |L0        |          |
|goto         |L1        |          |          |
|+            |p         |1         |t1        |
|=            |p         |t1        |          |
|goto         |L2        |          |          |
|+            |q         |1         |t2        |
|=            |q         |t2        |          |
```

```
Triples:

|    INDEX    |     OP      |   ARG1    |    ARG2   |
-----------------------------------------------------
|0            |<            |x          |y          |
|1            |if True goto|[0]        |L0         |
|2            |goto         |L1         |           |
|3            |+            |p          |1          |
|4            |=            |p          |[3]        |
|5            |goto         |L2         |           |
|6            |+            |q          |1          |
|7            |=            |q          |[6]        |

Indirect Triples:

|  ADDR  |    INDEX    |   |    INDEX    |     OP      |   ARG1    |    ARG2   |
-----------------------    -----------------------------------------------------
|31      |0            |   |0            |<            |x          |y          |
|32      |1            |   |1            |if True goto|[31]       |L0         |
|33      |2            |   |2            |goto         |L1         |           |
|34      |3            |   |3            |+            |p          |1          |
|35      |4            |   |4            |=            |p          |[34]       |
|36      |5            |   |5            |goto         |L2         |           |
|37      |6            |   |6            |+            |q          |1          |
|38      |7            |   |7            |=            |q          |[37]       |
PS C:\Users\vishw\Coding\Compiler-Lab\Week-10>
```

Result:

We have successfully implemented a C++ code to find Three Address Code and it's 3 data structures (quadruple, triple, indirect triple).